



CROSS PLATFORM DEVELOPMENT

CO2404



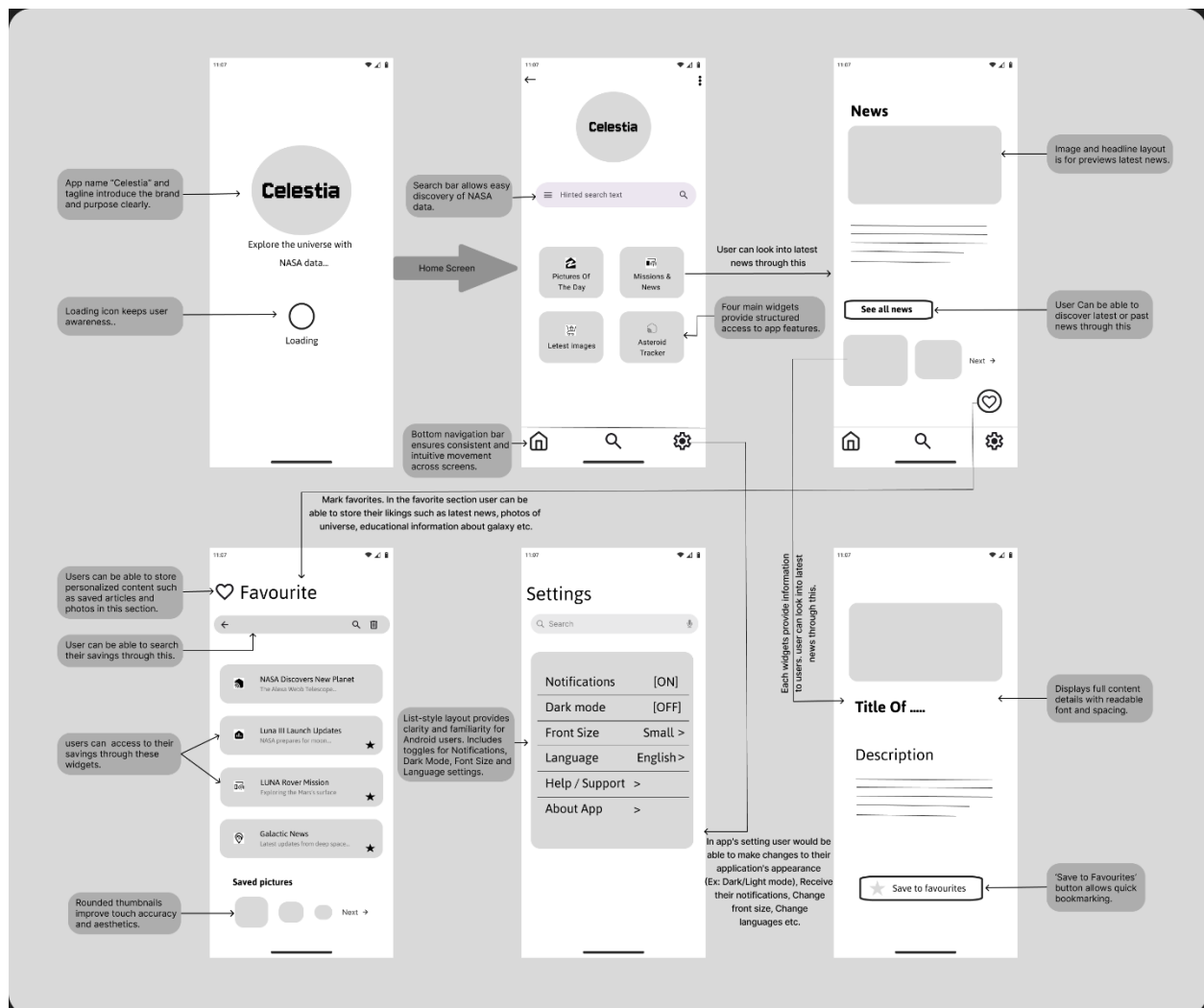
NAME: K.A.I.PIUMIKA

Student Number: G21328023

Overview

“CELESTIA” is a mobile application that taps into NASA’s open APIs to display live space content such as the astronomy pictures of the Day (APOD), different planets’ information and asteroid tracking data. The app is designed with cross-platform compatibility for both IOS and android users, prioritizing usability, accessibility and visual order in alignment with Human-Computer Interaction (HCI) principles.

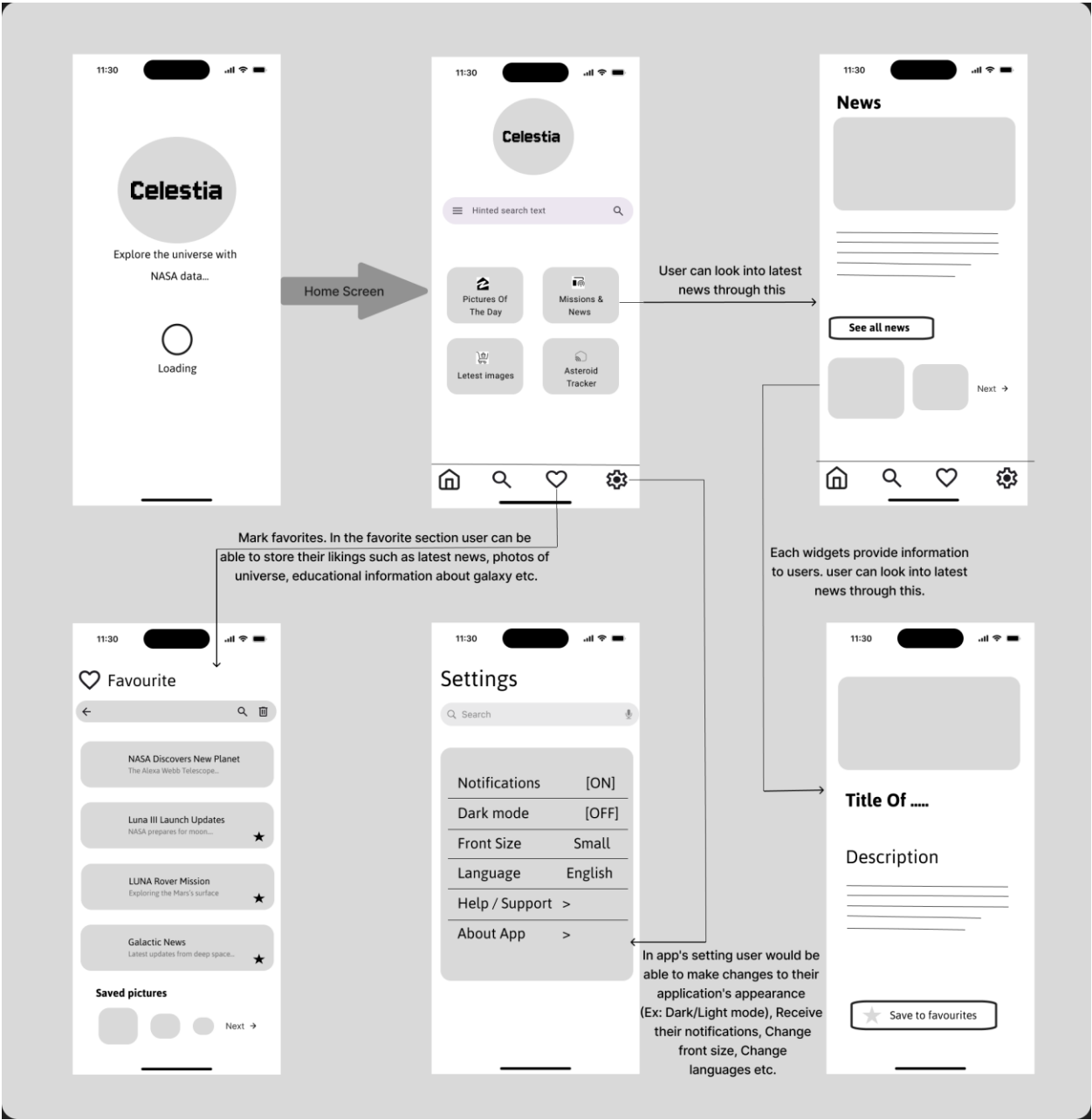
For Android:



Annotations - Android Version

- **Splash Screen:** Mainly introduces the app with little distraction while loading data from NASA.
- **Top App Bar:** Displays the app title and serves as a navigational anchor. And of course it is also to keep with user familiarity with Android.
- **Search Bar:** Here I Used a rounded search field with hint text to foster discoverability and minimize cognitive load. The more confusing instructions the harder your users' brains have to work.
- **Navigation Bar (Bottom):** Home, Favorites and Settings store icons follow the Android bottom navigation convention for quick access to high-level destinations.
- **Content Widgets:** The grid layout provides scan-ability and balance of white space (for example: Picture of the Day, Missions & News). I derived this idea from Gamble's "aesthetic and minimalist design" heuristic (Nielsen, 1990).
- **News Screen:** The news screen displays image header with captures the stories (title>summary>link) a "See All News" button signifying clear affordance to more information.
- **Favorites Screen:** Presented as a list. Each card promotes familiarity with icons for adding/removing item from favorites consistent with Norman's principle of visibility of system status.
- **Settings Screen:** Toggle switches and dropdowns allow user to manage notifications, dark mode, and language.

For IOS:



Annotations – IOS Version

- **Splash Screen:** Very similar as original android, but the design follows the iOS Human Interface Guidelines with a centered logo and minimal text; plus the status bar is consistent with the iOS colors in appearance ([Apple, 2024](#)).
- **Navigation (Tab Bar):** Again, this is akin to the original android introduction, however, the tab bar was placed at the bottom of the screen to help maintain clarity and conform to iOS standards. This also helps with familiarity, as other iPhone users are used to tab-based navigation.
- **Home Screen Layout:** This is similar the original android; but the home screen also maintains the centered logo and clean grid layout for the main functions, which keeps with the idea simple, and maintaining spacing from the edges, which accentuates the look and feel of Apple products.
- **Search Field:** This field looks very similar to the android introduction as well, with a rounded input bar that follows the native iOS control design, to enhance usability and accessibility in a way that is also intuitive.
- **Content Widgets:** The use of a rounded corners and insightful shadow produces a feeling of visual depth; which is consistent with other iOS products, and the overall style of iOS's "layered interface" design.
- **Favorites Screen:** Again, this screen is very similar to original android; but the use of iOS list cells produces greater style with consistency with iOS, while still conforming to the expectations of the platform.
- **Settings Screen:** This screen displays the grouped list items using the iOS toggle and chevron style, combined to display common options; this screen is designed so that users can easily recognize options instead of committing to memory so that they can engage in a quicker or more direct interaction (vs android the button means a different action on settings).
- **Accessibility Features:** High contrast text, voiceover compatible and features that conform to iOS accessibility guidelines, with large tappable areas help iOS adhere to the standards of WCAG AA compliance ([W3.org, 2024](#)).

❖ Accessibility Features

Accessibility is incorporated into the design to make it universally accessible.

Contrast & Colour:

The wireframe is a grayscale foundation to maximize the contrast between the text and background. Ultimately, the app will incorporate both dark and light theme options to benefit users with visual impairments or sensitivity to light.

Text Alternatives:

Each image component (i.e., APOD image or Mars Image) has alt text descriptions, so screen it can be read out loud to users who are visually impaired.

Scalability & Font Size:

The setting page has the option to change the font size, which allows users to adjust the font size for readability.

Touch Targets & Layout Spacing:

Buttons such as the "Save to Favourites" button are designed with large touch targets (minimum of 44x44 points), in adherence to Apple and Google accessibility guidelines.

Feedback and Clarity:

Interactive objects such as saving favourites or changing themes provide stimulation through visual and touch feedback, and lets users know that the desired outcome was achieved.

❖ Usability:

The interface of “**Celestia**” mobile application enables users to engage with a clear navigation structure. The bottom navigation bars across the platforms let users easily navigate between the Home, Favorites, and Settings screens in a way that decreases a user's cognitive "memory load" ([Nielsen, 1990](#)). And of course the user can be able to identify the representations in the navigation menu because they labeled with meaningful icons that the users can be able to recognize rather than remember. This is also a principle of usability ([Norman, 1983](#)).

❖ Consistency:

The app looks and feels familiar on both platforms because it follows their usual design styles that govern each OS, while also maintaining the brand identity. Material Design components like CardViews, Floating Action Buttons, and grid layout patterns were used for Android for sense of familiarity. On iOS, Apple common component interaction patterns were used and included tab bars, toggles, and rounded input fields. Thus, both versions managed to retain the feeling of being native users of their respective devices. This serves as one aspect of the internal consistency while keeping the view of an aesthetically consistent app design ([Apple, 2024](#)).

Human-Computer Interaction Principles:

The design process of “**Celestia**” mobile application came to my mind through Norman's (1983) principles. And of course I used various load symbols and reactive knob patterns to show users the system status. The user control and freedom aspects were encapsulated within the structure of the Favourites and Settings screens, where users can modify preferences and personalize the appearance however they want.

Overall, the Celestia app describes a user centered design while remaining true to cross platform-specific expectations not sacrificing usability or design principles. Through HCI and accessibility principles, the design allows for all users to explore NASA's data naturally and inclusively.

Development Log

Date	08 Feb 2026
Time spent	5 hours
Tasks completed	• Used MultiProvider to create application's primary entry point and provide for structuring the state of the application. • Used ChangeNotifier to connect ThemeProvider and FavouritesProvider. • Create a SplashScreen using video_player package to display an animated introduction. • Create asynchronous initialization within initState, assuring that video assets are loaded properly at the time of being used. • Automatically navigate to main application interface using Navigator.pushReplacement after splash animation finished playing. • Organized project folder structure to separate screens, providers and widgets.
Challenge/s	• Safe navigation could occur within the listener of video controller. • Asynchronous video initialization did not cause UI blocking. • Memory leak prevention via proper lifecycle management.
Solution/s	• Video playback was delayed until after video loaded by using initialize().then(). • A listener was implemented using VideoPlayerController to check for completion and error detection on completion of video. • Overridden dispose() to release controller resources properly.
Reflection	• Architectural foundations of the application were created during this session. Provider provided for scalable state management, which adhered to separation of concerns principles. The splash screen adds to usability and helps to create a high-quality first impression of the application using modern mobile application standards.
Next steps	• Create Home Dashboard interface. • Integrate into NASA API endpoints.

Date	12 Feb 2026
Time spent	5 hours
Tasks completed	• The home screen dashboard interface was completed by designing and implementing the following: • A grid view layout was created to hold the feature navigation tiles. • Included animated glowing logo through AnimationController, Tween, and CurvedAnimation. • Glassmorphism UI design was incorporated into the interface using BackdropFilter as a visual enhancement.

	• Created a reusable buildTile() method to help with maintainability and reduce code duplication. • Configured navigation between feature screens using Navigator.push().
Challenge/s	• Correctly implement animation lifecycle management using SingleTickerProviderStateMixin. • Maintain a responsive layout across multiple screen dimensions.
Solution/s	• Created glowing animation using repeat(reverse: true) to allow for smooth transitions. • Created consistent spacing/padding within grid view for balanced visuals.
Reflection	• This phase focused on user interface design principles such as visual hierarchy, consistency, and feedback. The use of reusable components increased modularity while following the DRY (Don't Repeat Yourself) principle.
Next steps	• Integrate NASA "Astronomy Picture of the Day" API.

Date	13 Feb 2026
Time spent	5 hours
Tasks completed	• Created a connection to NASA's Astronomy Picture of the Day (APOD) API through HTTP GET requests. • Used FutureBuilder widget to render queried data asynchronously. • Parsed the JSON response from the APOD API by calling jsonDecode() function. • Used CachedNetworkImage to render remote images, resulting in better overall application performance and increased image-caching efficiency. • Implemented Hero animations to allow for smoother transitions of images. • Added an ability to mark images as favourites through the use of provider state management.
Challenge/s	• Dealing with nested JSON objects returned by the APOD API. • Preventing duplicate entries in the favourites list. • Handling null-safety issues.
Solution/s	• Mapped/structured the JSON fields from the incoming response into a Map object. • Developed a method (isFavourite()) to check for duplicate entries in the favourites list by comparing the URLs of each image. • Used snapshot.hasData property of the snapshot object to check whether data was present and render accordingly.

Reflection	• This was beneficial in developing an understanding of integrating RESTful APIs using Flutter's FutureBuilder widget. The FutureBuilder greatly simplifies how you update the UI for asynchronous data return while still providing a timely manner of refreshing the UI.
Next steps	• Build out views for missions and more detailed news.

Date	18 Feb 2026
Time spent	5 hours
Tasks completed	• User a count parameter to fetch multiple results when implementing the MissionsScreen via the NASA API. • Create a NewsDetailScreen that will allow users to view additional content. • Use ListView.builder() to create a dynamic list that is efficient to render. • Use GridView.builder() for displaying content such as images in a grid format. • Pass structured JSON databetween screens with the help of Navigator routing. • Add favourite toggle functionality to both the list and detail views.
Challenge/s	• Handling dynamic JSON arrays. • Keeping the UI consistent across multiple screens. • Rendering layouts with high image use efficiently.
Solution/s	• Create typed lists by converting JSON Arrays into List. • Reuse card and listTile styling to create consistency in look-and-feel amongst all widgets. • Limit the amount of data rendered when possible to lessen the overall processing load.
Reflection	• After completing this stage, I have gained an appreciation for how to effectively render dynamic data and build widgets efficiently using less memory. Using builder patterns will increase performance by only rendering items that are visiblle on the screen.
Next steps	• Add Search capability. • Create Aseroid Tracking.

Date	20 Feb 2026
Time spent	3 hours
Tasks completed	• Built a SearchScreen with TextEditingController support/rendering • Connected NASA Image Search APIs using dynamic query parameters • Included a loader that adjusts based on boolean flags • Built an AstroScreen from NASA NeoWs API • Obtained nested information (diameter, close approach date) from asteroids • Used toStringAsFixed() on numerics for formatting
Challenge/s	• Extracted information from deeply nested JSON objects • Handled cases when users perform empty searches • Avoided too many API calls
Solution/s	• Accessed the keys in the JSON file using a logical approach, preventing run-time issues caused by accessing incorrectly • Included digesting checks on returning from empty queries • Limited the amount of result returned by search to improve performance
Reflection	• Successfully accessing deeply nested JSON APIs enhanced problem-solving skills related to data parsing in JSON. Real-time searching enhances user-friendliness and engagement.
Next steps	• Implement persistent storage & settings.

Date	21 Feb 2026
Time spent	4 hours
Tasks completed	• Completed SettingsScreen with toggle between themed/favorite content on screen. • Implemented ThemeProvider with switching between ThemeMode options. • Completed FavouritesScreen to show contents saved as favourite items. • Implemented SharedPreferences for persistent local storage of saved favourite items. • Encoded/decoded items adding to favourites list using JSON Serialization. • Performed full end-to-end testing and fixing bugs in the application.
Challenge/s	• Being able to load persistent data correctly, when restarting an app. • Updating the UI reactively after data has changed.
Solution/s	• Access persistent local storage using SharedPreferences.getInstance(). • After every update made, call notifyListeners() to update the user interface.

Reflection	• Local persistence enhances the user experience by maintaining the user's preference across sessions. Using Provider architecture for state management ensures that the codebase is well structured and scalable.
Next steps	• Improve the error handling strategies in the application. • Improve the overall aesthetics and responsiveness of the user interface.

Date	23 Feb 2026
Time spent	4 hours
Tasks completed	• Refactored API requests codebase for improved readability and maintainability. • Developed a standard HTTP request framework across all API-integrated screens. • Enhanced error handling by adding a check of response.statusCode prior to decoding JSON responses. • Developed a fallback user interface (UI) that displays a message when the API returned no data. • Enhanced null-safety checks for deep-nested JSON fields
Challenge/s	• Certain API calls returned unexpected structures when requested (e.g. missing data). • The UI would crash (via an "unhandled exception") if a required deep-nested field was missing.
Solution/s	• Implemented conditional logic patterns to check whether the property was valid before accessing nested properties. • Added a generic user-friendly message when deep-nested fields were null. • Validated all JSON responses' status code is 200 before parsing their content.
Reflection	• This experience has made the application more robust and reliable, and the developer implemented "defensive coding" to minimize the likelihood of runtime exceptions while maximizing the user's ability to use the application, especially when using external APIs.
Next steps	• To improve widget build performance. • To improve UI responsiveness.

Date	24 Feb 2026
Time spent	4 hours
Tasks completed	• Investigated how the Application consumes Widget Rebuilds through Providers. • Limited the Listening Scope of Provider to eliminate unnecessary UI Paints. • Separated UI Render Logic from Data Fetching Logic. • Reduced the number of nested heavy Widgets in the ListView to improve ListView performance. • Completed Performance Tests on an Emulator.
Challenge/s	• Unexpectedly high Rebuilds on certain screens. • Degraded Performance on Image Heavy Grids.
Solution/s	• Utilized listen: false on UI Updates where there were no UI Updates. • Reduced Widget Tree Depth in Grid Items. • Verified that Cache Mechanisms were in use.
Reflection	• Understanding Widget Rebuild Cycles is necessary for Flutter Optimization. Proper State Management will help avoid Performance Bottlenecks in larger Applications.
Next steps	• Develop Consistent Navigation Transitions. • Enhance Ease of Development from Animation Visual Aspect/Feel to Animation Consistency.

Date	26 Feb 2026
Time spent	3 hours
Tasks completed	• Enhanced the consistency of the navigation flow from screen to screen • Standardized all AppBar's style across all screens • Ensured all Card's elevation and border radius are consistent • Enhanced the consistency of the Hero animation across all image-based screens • Tested the navigation stack, ensuring that back navigation will not happen accidentally
Challenge/s	• Transitions between some screens were visually inconsistent • Some back navigation would navigate to unwanted screen(s) in some areas of the navigation flow
Solution/s	• Created a standardized approach for navigation via MaterialPageRoute • Used pushReplacement for the SplashScreen to prevent back navigation • Reviewed the logic of the Navigator stack
Reflection	• It is important to have a consistent navigation experience as it enhances the usability of the application while adhering to HCI principles of predictability and feedback.

Next steps	• Improving local storage management • Implementing structured validation of the data.
------------	---

Date	27 Feb 2026
Time spent	4 hours
Tasks completed	• Improved SharedPreferences functionality. • Implemented a validation check to eliminate duplicate entries when saving favourite items. • Successfully confirmed persistence of favourite items after the user restarts the app. • Enhanced JSON serialisation and deserialisation logic. • Confirmed that the ThemeProvider state updates successfully across different screens.
Challenge/s	• Ensure that favourite items remain in sync after an app is restarted. • Process the decoding of JSON into correct data structures.
Solution/s	• Controlled the flow of update notifications before signalling notifyListeners(). • Verified the data before encoding and saving it to avoid the possibility of duplicate entries.
Reflection	• The need for persistent storage is to create a seamless transition from one part of the program to another, increasing both the continuity of how users interact with the application and the overall reliability of the programme. This phase has provided a deeper knowledge base of how to integrate local storage into applications built using Flutter.
Next steps	• Full-system testing of the application • Identifying any potential edge cases of the application.

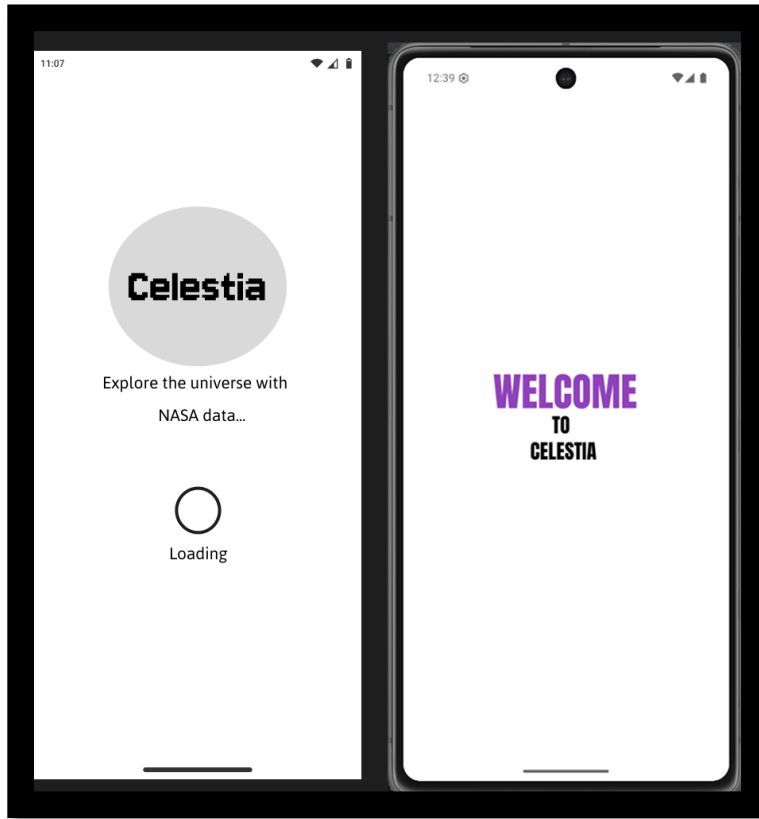
Date	28 Feb 2026
Time spent	2 hours
Tasks completed	• Functional testing of all screens was accomplished. • API connectivity was tested with various network conditions. • Error handling was verified for slow and/or failed networks. • UI response time testing was done across all emulator sizes. • Code structure and documentation clarity were reviewed.
Challenge/s	• Ability to simulate network latency for real world testing scenarios. • Difficulty in finding minor UI positioning differences.

Solution/s	• Network speed was manually throttled during emulator testing. • Padding/spacing was adjusted in order to have consistent layout.
Reflection	• Overall the Systems Testing supported verifying that the application worked as an integrated system; therefore testing under all scenarios is necessary and required before final production release.
Next steps	• Ready for final documentation. • Finalize all diagrams and technical report sections.

Evaluation

Splash Screen (Celestia App)

Wireframe vs. Final Implementation



Analysis of Wireframe Vs Implementation

- Original Wireframe
- Static and Simple
- Circular logo inside the application logo
- Tagline: “Explore NASA Data and Satisfy Your Curious Spirit...”
- Indicating loading process (progress) - Loading
- Background is just a colour (white).

Final Flutter Implementation

As a result, there was a significant upgrade in both presentational and intrinsic characteristics/functionality from the original wireframe. This was due to the introduction of an animated video that provided a new experience (i.e., new type of content, a distinctive branded video). The completion of the animated video led to the automatic transition into the application (from splash screen → main application). Additionally, there was a safety net established for error handling should the video not load (directly navigating users into their main applications). Finally, the information displayed to users when they entered into the splash screen (the circular logo, loading icon, etc.) was simplified (cleaned up) to create a more professional appearance and first-time user experience by utilizing motion graphics as the way of creating this animation.

The change from a static splash/loading screen to a video-based animated splash creates a much greater perceived level of quality and enhances the end user's experience and the overall quality of the application.

Evaluation Summary Overall

The splash screen has transformed from a simple loading screen to a professionally branded animated welcome message. The wireframe was different in the following ways:

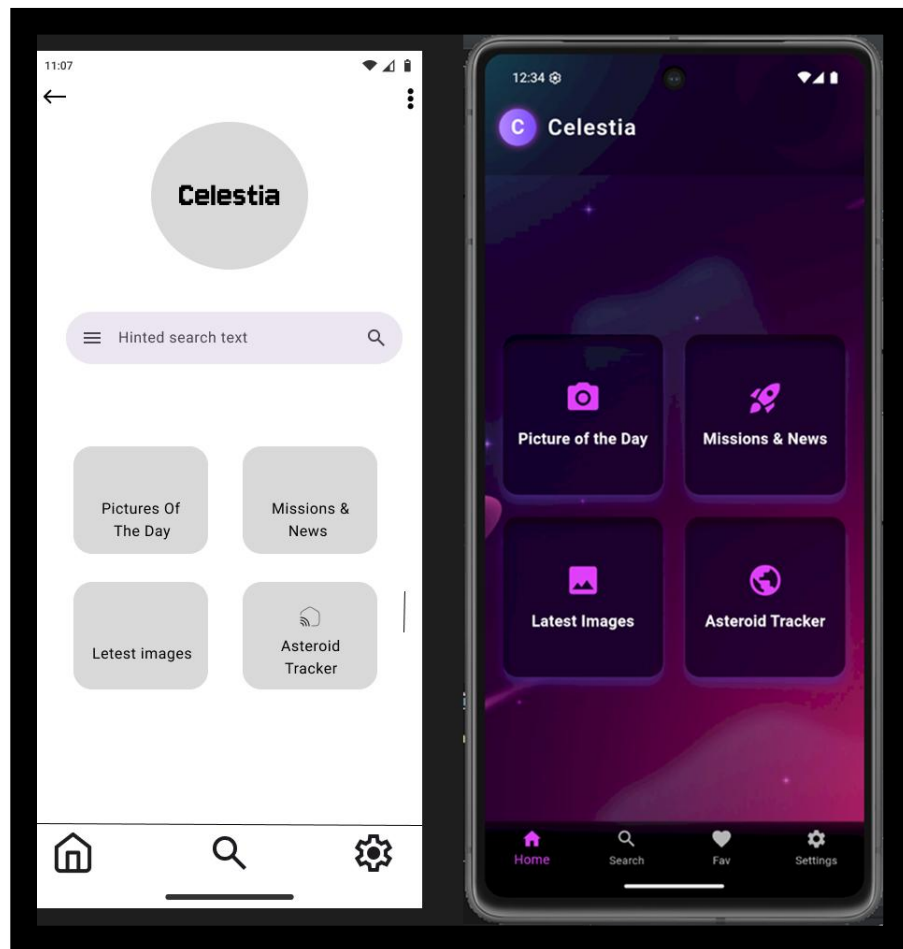
- Improved visual quality
- Improved level of engagement
- Improved perceived application value
- Improved technical robustness

From a usability standpoint, the splash screen is effective in:

- Aesthetic design
- Error prevention
- System status visibility
- Brand reinforcement

Home Screen (Celestia App)

Wireframe vs. Final Implementation



Wireframe vs Final Version Reflection

The Home Screen in Flutter went through significant change between the original wireframe and the final UI/UX design. Structurally, the final Home screen was kept consistent with a grid navigational layout and four main feature cards. Significant changes occurred in the aesthetics and the way users interacted with the Home screen were introduced as part of the development process.

Key Similarities:

- 2x2 Grid layout structure was kept consistent
- All four main features remained the same:
 - I. Picture of the Day
 - II. Missions & News
 - III. Most recent images
 - IV. Asteroid Tracker
- Acted as the main navigation for users, on this screen they can navigate around the Celestia App.
- Each feature was accessible by tapping on it (each feature is presented as a card that can be tapped on).

Key Enhancements/Differences:

- A glass-morphic style AppBar was created through the use of **BackdropFilter** with blur effects.
- A custom animated logo (that glows) was produced using **AnimationControllers** and **Tweens** was not a part of what was presented in the wireframe design.
- A background image that represents space was added for added immersion into the theme.
- Cards were modified to contain:
 - Rounded corners
 - Shadow elevations
 - Semi-transparent dark backgrounds
 - Purple accent highlights
- Icons were added to provide more ease of visual recognition for users.
- Smooth navigation to users through `Navigator.push()` for navigating to different screens of the App.

With these enhancements, the wire-frame became a static experience into a visually dynamic, user interactive experience with the space exploration theme incorporated.

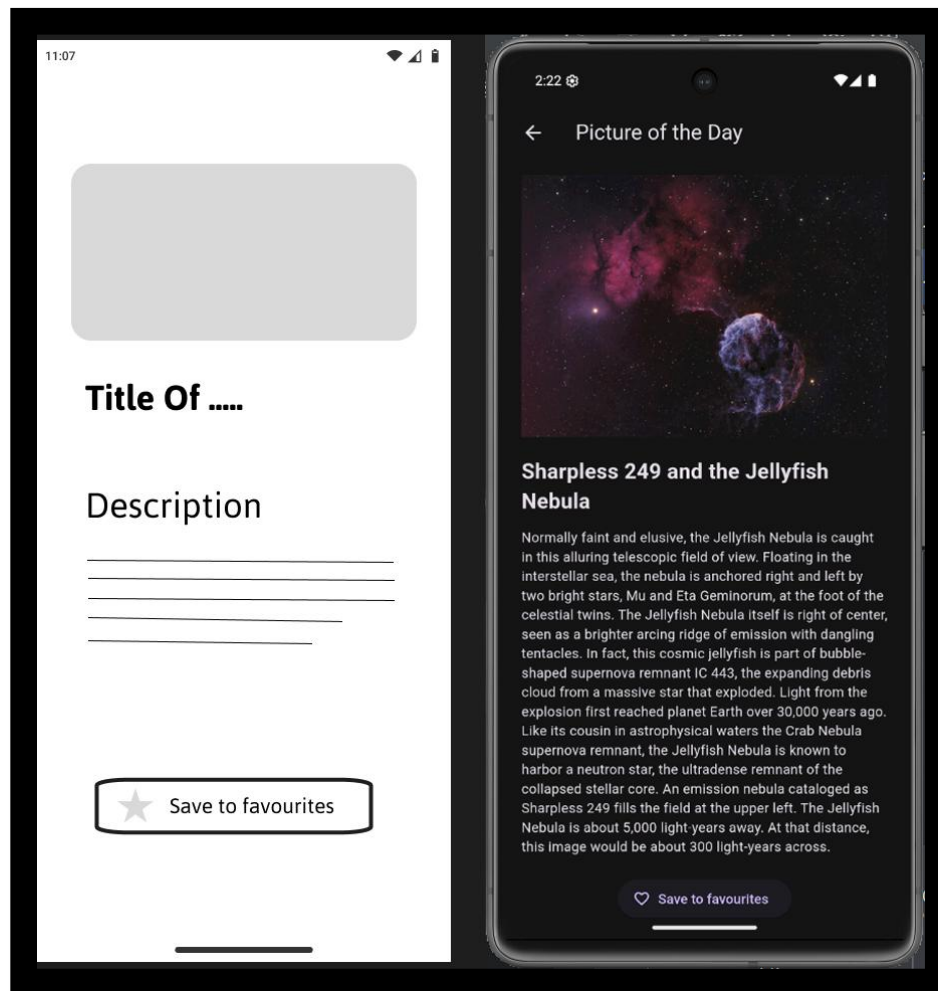
Evaluation Summary as a Whole

The final home screen design enhances the wire frame created at the beginning of the project by adding many new features. Those features include a thematic visual identity; smooth navigation between areas on the home screen; animations to help engage with other users of the app; a strong visual hierarchy for the app's content; and a consistent layout throughout all areas of the application.

Usability-wise, in terms of meeting Nielsen's usability principles, the design performs very well. The usability principles that are met include: (1) Aesthetic design; (2) Recognition instead of recalling an item; and, (3) Consistency.

Picture of The Day Screen (Celestia App)

Wireframe vs. Final Implementation



Comparing the wireframe to the final implementation, we can observe that the final Adobe Flutter based implementation was significantly enhanced in terms of functionality, interactivity, and visual richness, while still maintaining the core functionality of the original wireframe. The wireframe only contained a sample of what a featured image will look like, plus a title and a description of that image and a button to save to favourites, whereas the final implementation resulted in a fully functioning and dynamically created screen with data being downloaded and/or uploaded from NASA's APOD API.

In addition, the wireframe did not contain any functioning links or buttons. The final implementation has enabled link interactions using the newly implemented dynamic content and provides a clear indication that a user action occurred by way of visually appropriate feedback mechanisms and through the use of transition animations.

Key Similarities

- The original vertical layout of the wireframe has been preserved.
- The screen has a single piece of primary content, which will reduce a user's cognitive overload when viewing the content.
- The original purpose of the screen as a place for a user to view the daily featured image has been maintained.
- The wireframe design had a minimalist approach and this same minimalist design approach was used in the final implementation.

Key Enhancements/Differences:

- Live Integration of NASA API replaces the original static wireframe that was only a placeholder image.
- Utilizing **FutureBuilder** allows data to be retrieved asynchronously.
- A loading indicator - **CircularProgressIndicator** - provides user feedback while the API call is made.
- **CachedNetworkImage** allows less network usage and better performance for retrieving images from the internet as they become available.
- A Hero Animation provides seamless movement from one screen (the Home screen) to the other (the Picture of the Day).
- Using the Provider for state management in saving pictures to your favourites is an efficient way to maintain state.
- The icon switching between outlined and filled heart provides visual feedback on whether an item is saved as a favourite.
- The use of a **SingleChildScrollView** allows scrolling through many lengthy descriptions of pictures without the user losing their place on the screen.

These improvements have transformed this wireframe into an interactive production quality screen.

Evaluation of the Picture of the Day Screen

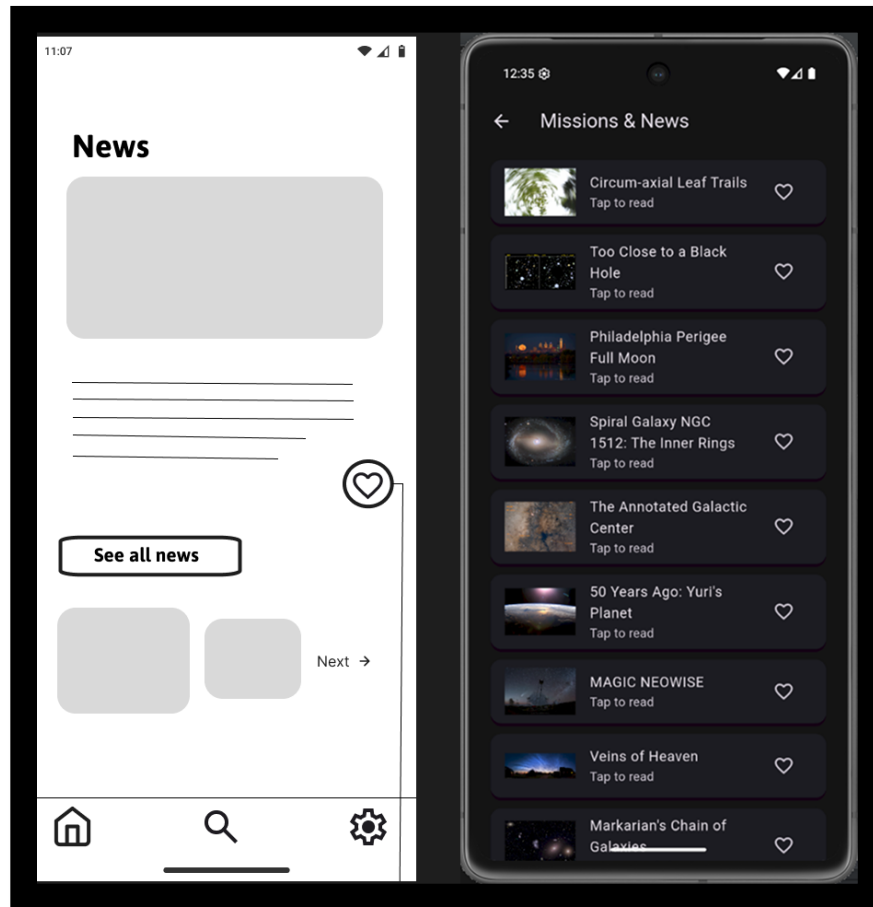
The Picture of the Day Screen is a well-developed feature that provides an excellent basis for an effective end-use product by:

- 1) Presenting content clearly.
- 2) Handling data asynchronously.
- 3) Providing a visually appealing design.
- 4) Allowing you to maintain record of your favourite items.
- 5) Being in line with usability heuristics (as defined by Nielsen).

Other than minor enhancements potential in the areas of error handling, advanced interaction, and accessibility, this implementation of the Picture of the Day Screen balances usability, appearance and technical characteristics to be a very well designed functional component of the Celestia Flutter Application.

Missions & News Screen (Celestia App)

Wireframe vs. Final Implementation



The finished Flutter version of the Missions & News screen represents an evolution from the original wireframe into a working, data-driven user interface. The wireframe primarily showed the based news-layout preview format, with image, headline, short description and navigation affordances being used to create this new screen. In the end product, this concept was transformed into a vertically scrollable list of news items that each delivered users to a detail view of an individual item.

The wireframe proposed two different means of interacting with the news content: “See all of the news” and pages of news items. The final product actually allows the user to interact with the news content in a single list format that can

be navigated through by only scrolling down the page; therefore, improving the ability to use this screen as well as simplifying the interface for the user.

Key Similarities

- The news items are provided in a list format.
- Each news item contains:
 - a thumbnail image,
 - a title/headline, and a description
- The user can touch an item of news to see details about it.
- There is a clear distinction between the news list and the detailed view.
- The screen serves as a central location for astronomy related information/news.

Important Improvements and Changes Made

- Live NASA API data now replaces static wireframe placeholders.
- **FutureBuilder** allows you to retrieve data asynchronously instead of synchronously (i.e., waiting for one call to complete before making the next one).
- Material List Tile widgets provide an easy-to-use, consistent and accessible way to display the news articles.
- Navigation to a dedicated News Detail page is accomplished with **Navigator.push()**.
- The News detail page has:
 - Full-size Image
 - Complete article detail
 - Ability to bookmark article through use of a Provider.
- Improved dark theme style enhances readability and maintains the visual identity of our application.

These changes convert the wireframe from conceptual design to a fully functional and maintainable feature.

Overall Evaluation and Summary of Findings

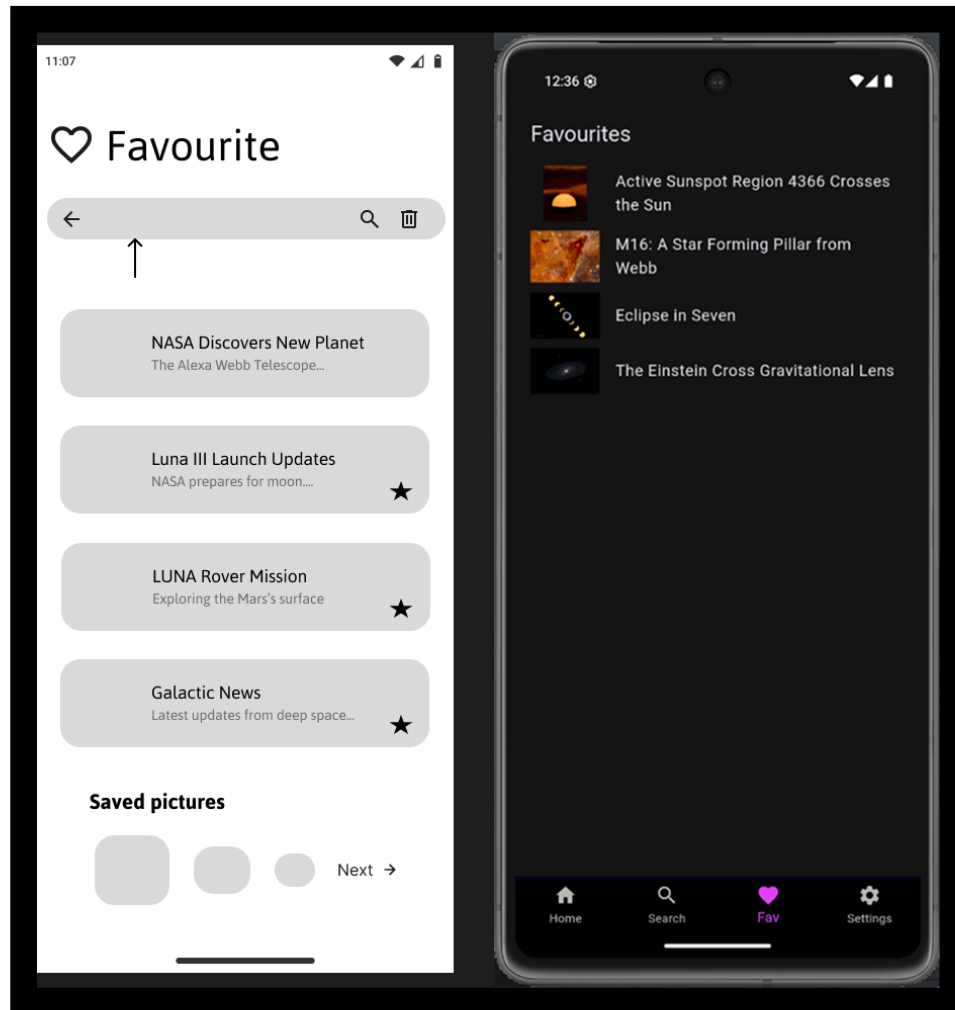
The way the Missions & News page converted a conceptual wireframe into a working, functional, and usable Flutter component consistent with modern mobile usability practices is noteworthy. The page has many strengths, including:

- Clear information hierarchy
- Simple & intuitive navigation between content
- Consistency in interaction patterns
- Asynchronous loading of data.
- Favourites integration is done without any issues.

Some areas that could be improved include: error handling, content scaling, and advanced user controls. However, overall, there is a strong foundation of UI/UX principles, Flutter architecture, and usability heuristics; therefore, it will be a very strong and well implemented component to the Celestia application.

Favourites Screen (Celestia App)

Wireframe vs. Final Implementation



Although there were changes to the wireframe to create the final Flutter implementation of the Favourites screen, the core purpose is realised and user interaction simplified, while achieving a more maintainable product. The original wireframe suggested creating a central hub for the user's saved items and would include the following: search functionality, pagination functionality, an area for displaying saved news, and an area for displaying saved pictures; all broken down into separate sections of the favourites hub. The final implementation is a single, vertically scrolling list of the user's favourites that concentrates on both being clear to the user and creating a simple user experience as possible.

Although some of the more advanced features in the wireframe (search bar, pagination, and category separation) were not implemented in the final design, the overall functionality of the design met the primary requirement of the functionality, which is the ability of the user to store and retrieve saved items.

Key Points of Similarity:

- The user accesses the Favourites screen to store all his/her saved items in the app
- The user can see the following information about each saved item:
 - A thumbnail image of the item
 - The title of the item
- Each favourite item is displayed in a vertical list format
- The Favourites screen is centred around the heart/favourite concept
- The Favourites screen can be accessed from any of the other screens in the App through the persistent navigation component.

The following improvements and New Features are included with this release:

- Replaced Static Wireframe Placeholders with Live Saved Content
- Introduced Persistent Storage using SharedPreferences
- Implemented Global State Management using Provider
- Created Dynamic Lists using **ListView.builder**
- Added Automatic UI Updates for when Favourites Are Added or Removed
- The Dark Theme has been Styled to be Consistent with the rest of the App.

These Enhancements Shifted This Screen from Simply a Conceptual Design to a Fully Functional, Data Driven Feature That Will Help Improve Long Term User Engagement.

Evaluation Summary

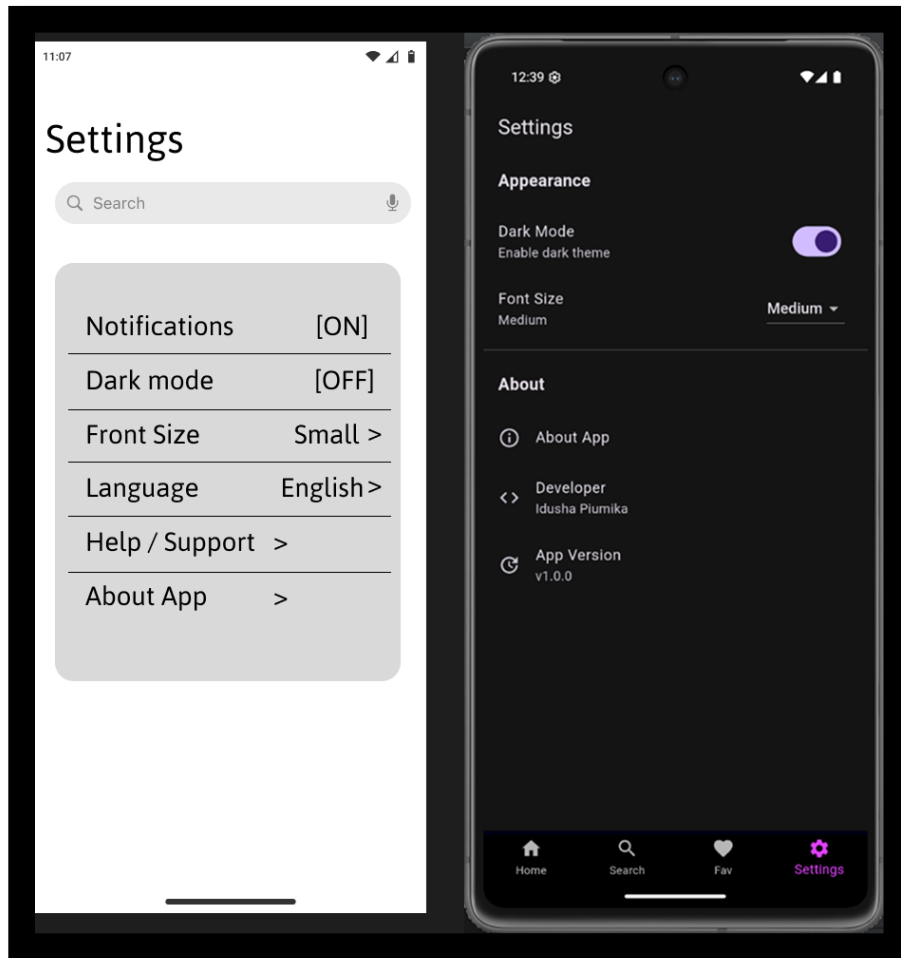
The favourites screen serves its purpose due diligence in allowing users to create, save, and return to preferred pieces of content. Its users will find the following usability principles have helped guide the design of the screen:

- Assistance with remembering rather than recalling (recognition vs. recall).
- Consistent user interface design.
- Supports minimalistic screen design.
- Users have control over information by using a persistent state.

The final product does not replicate the wireframe exactly, however, it does replicate what the wireframe intended to convey using a high-level of usability. By making a minor addition to the Favourites screen such as ability to search for Favourites, ability to delete Favourites and an empty state that has instructions for first time users, will allow this screen to reach a production ready level.

Settings Screen (Celestia App)

Wireframe vs. Final Implementation



Reflection on Wireframe vs Final Implementation

The Settings Screen final design clearly evolved from the original wireframe concept. Even though the overall concept of a structured settings view list remained the same, there were many additional changes made both functionally and visually throughout the development process.

Wireframe Characteristics

The original wireframe included the following:

- Search box
- Notifications toggle
- Dark mode Toggle
- Font size picker option
- Language selection option
- Help/Support selector
- About the app option
- Standard card-style/container layout
- Light theme appearance

Final Implementation Characteristics

This implemented screen displays the following new features:

- Structured Sections: “Appearance”, “About”
- Dark Mode Toggle (fully functional through Provider)
- Dynamic Font Size (via Dropdown Button)
- About App Dialog
- Developer Information
- App Version displayed
- Integrated Bottom Navigation Bar
- Adaptive Dark Theme Style

Key Similarities

- Vertical List view design for settings layout.
- Toggle interaction for Theme control.
- Font size option accessible
- About the app section providing details of the app.
- Simple minimalistic layout structure.

Improvements and Changes

1) The wireframe was to contain features such as:

- Notifications
- Choosing a language
- Help/Support
- The ability to search

These features weren't realized in the final build so the actual screen is simpler and does not show any features that are only partially functioning.

As a result:

The app is much clearer and not affected by functionality as there is nothing that is incomplete.

2) Consistent Theme

The final UI uses a dynamic dark theme and follows the application theme.

- Has been implemented correctly using the Material dark theme
- Purple will be used as the accent colour
- Typography is consistent with rest of application

The wireframe was just a visually neutral static image.

3) State Management For All Functionalities

The final implementation uses:

- The Provider for handling the theme
- Theme Mode to switch
- Dynamic scaling of the text

The final screen will display the application's theme and font size changes instantly unlike the static wireframe.

4) Grouping of Sections in a Structured Way

The final design has separated the following sections:

- Appearance
- About

This has enhanced the information architecture by separating the sections from each other whereas previously they were combined in the one section as found in the wireframe.

5) The About Section is now Dialog-Based

The 'About App' now utilizes:

```
showDialog(  
    context: context,  
    builder: (context) => AlertDialog(...)  
);
```

This has greatly improved the interaction design from the simple navigation arrows that were originally provided in the wireframe.

Overall Breakdown of the Evaluation

The final Settings Screen transformed from the wireframe was:

- Fully functional
- Theme-aware
- State managed
- Clean and Minimal
- Configuration screen that is easy to navigate

Key Improvements to Wireframe:

- Ability for the user to switch themes dynamically
- Can now scale font in real-time
- About screen now shows through a dialog
- Thematic dark modes should be consistent with if you're using one or the other
- Bottom navigation now integrated with rest of the app
- Completely adheres to the Material Design guidelines

Human Usability Guidelines Strengths:

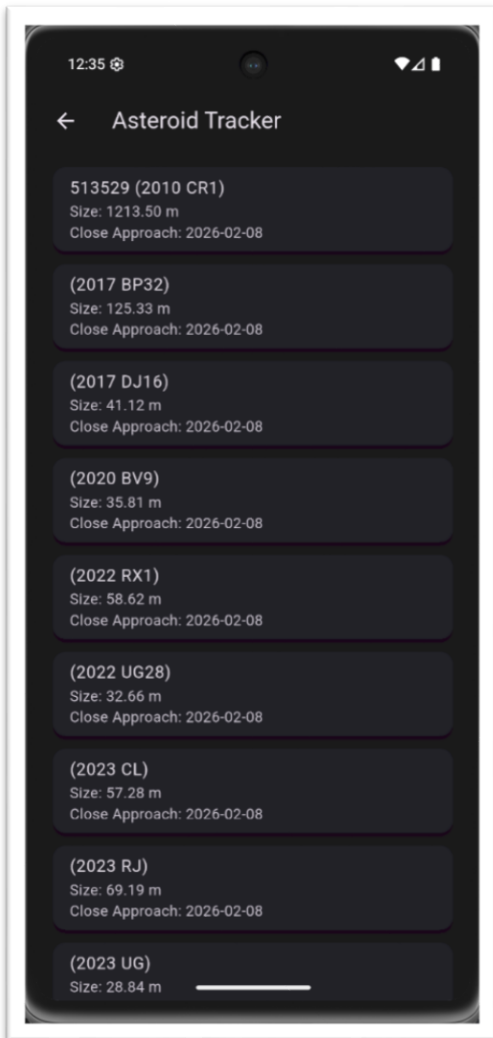
- Consistency is High
- Cognitive Load Is Low
- System Feedback Is Clear
- Strong Alignment To Aesthetic Value
- Error Potential Is Minimal

Potential Areas to Improve for Next Version:

- Add Notifications feature (like in wireframe)
- Add Language selection
- Add Help/Support section
- Add Reset to Default button

- Add subtle interaction animations
- Add confirmation of any persistent storage

Asteroid Tracker Screen (Celestia App)



Summary

The Asteroid Tracker Screen is able to dynamically retrieve current data from NASA's Near-Earth Object API, and display asteroids in an organized list format; unlike other screens, no wireframe was made first so this evaluation will focus on the quality of implementation, principles of usability, and the technical robustness of the screen itself.

Functional Review

Asteroids are retrieved via:

- **http.get()**
- **JSON decode**
- **Future-based async programming**
- **FutureBuilder for UI rendering**

And the following data is displayed:

- Asteroid name
- Estimated largest diameter in meters
- Date of close approach

The layout of the screen is implemented using:

- **ListView.builder**
- **Elevated Card widgets**
- **Rounded corners to maintain a modern UI style.**

Asteroid Tracker Screen Summary

The Asteroid Tracker screen works great and gives instant data on NASA asteroids. The layout of the list is easy to read, and the user can scroll through the asteroid info without lag. There is a loading symbol on the screen which lets the user know when data will be available.

The design of this screen is straightforward and in line with the rest of the app. Asteroids are presented in cards, which help to keep the info neat and organized.

While the screens perform as intended, there is no feedback provided (displaying an error message) when the user is offline (in other words, when there is a internet failure). There is also no ability to click a refresh button or apply filters to the list. By adding these features to the screen, the user experience could be enhanced further.

In summary, the Asteroid Tracker screen is functional, clear, and technically sound; however, it could benefit from adding functionality for handling errors and for providing the user with additional control options over their experience.

+ Latest Images Screen (Celestia App)



Summary

This screen displays images sourced from NASA's Image API in a grid format using:

- **GridView.builder**
- **CachedNetworkImage**
- Hero animations for transitions

Overall Evaluation - Latest Images Screen

The "Latest Images" screen is very visually appealing and interactive. The use of a grid layout allows users to easily s

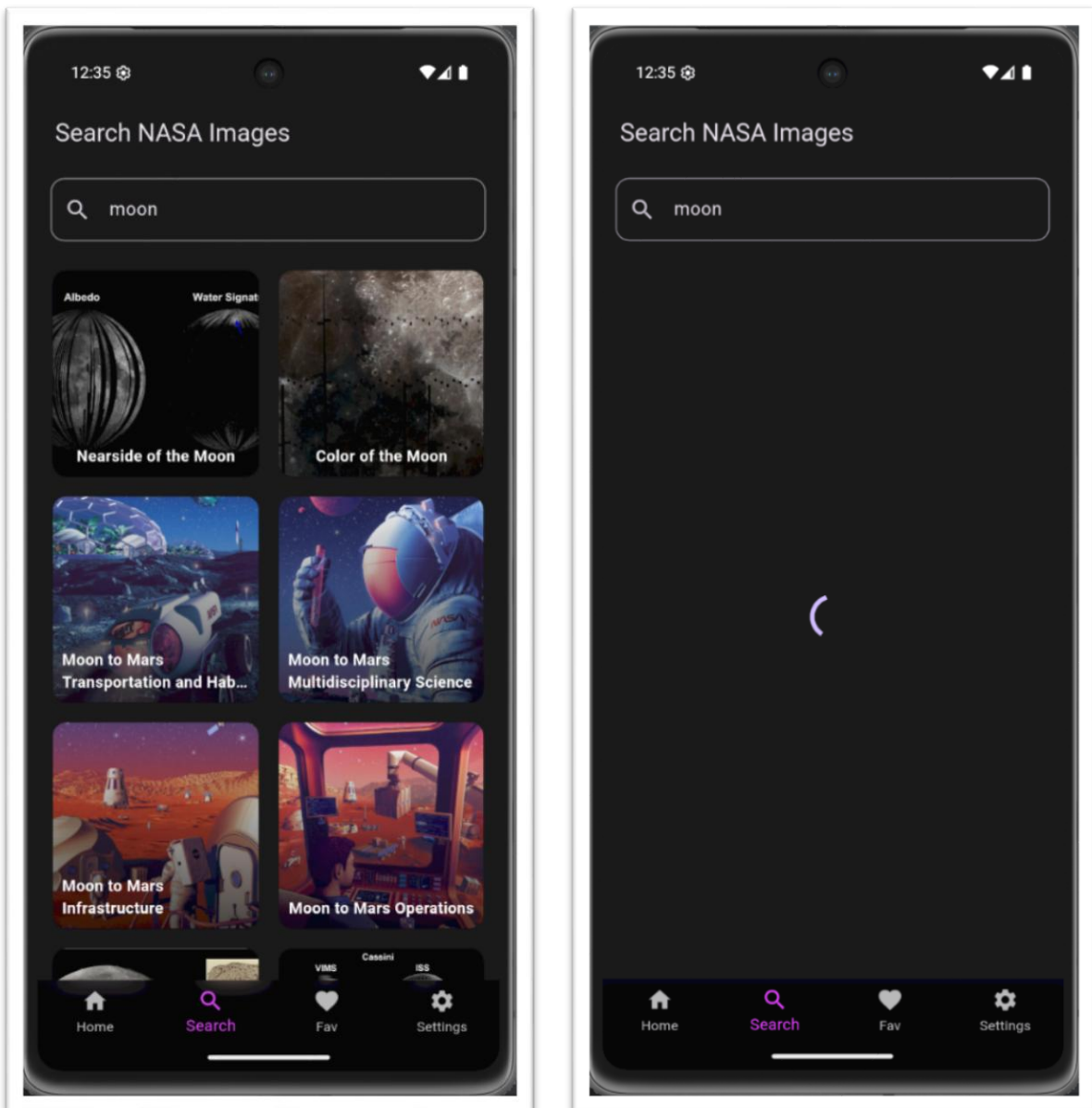
ee NASA's images. By utilizing image caching users experience an increase in speed when loading images because of the caching being used.

Navigation from the "Latest Images" screen to the "Detail" screen works well and feels clean and modern. Also, the use of loading indicators lets users know that there is something loading before it appears on the screen.

However, other than that if an image doesn't load, there isn't a message or placeholder to let the user know that the image has failed to load properly, nor are there any search or filter features available on the latest images screen.

Overall the "latest images" screen is the best visual design within the app. It is visually appealing, is responsive, and is user-friendly; however, it could benefit from having better error handling.

Search Screen (Celestia App)



Overview

The Search Screen allows users to search NASA images dynamically.

The Search Screen includes:

- TextField with onSubmitted
- Loading state management
- Conditional rendering
- **GridView** for search results
- Gradient text overlay

Overall Assessment – Search Screen

The Search Screen Incorporates an Interactive Functionality within the Application. This Allows Users to Input a Keyword to Find NASA Based Images with Ease. The Search Field is Simple and Easily Readable.

There are a variety of Different states displayed on the Search Screen:

- A Loading Indicator while Searching for Images
- A Message Displaying No Sheets Can Be Found If No Search Has Occurred
- A Grid Displaying Results When There Are Results

The Overall Layout of the Search Screen is Clean and Consistent with the General Overall Design of the Application, and the Gradient on the Image Title Will Assist Users with Legibility.

There Is No Error Message Displayed When a Search Does Not Produce Results nor is there a Quick way to Clear the Search Field for a quick way to perform new searches.

Overall, The Search Screen Provides A Functional, User-Friendly, and Variably Flexible Feature within the Application but there Are Opportunities for Improvement.

Heuristic Evaluation of the Celestia Mobile App

(Based on Nielsen's 10 Usability Heuristics – Nielsen, 1990)

1) Visibility of the system's status

- Strengths: Users are provided with consistent feedback throughout the application via loading indicators (spinners for API calls and image loading state, splash transition). Users receive immediate feedback when they have made dynamic updates like favourites or theme changes, thus providing proof of activity occurring via system.
- Weakness: Error messages or failed states when loading an API are rarely communicated in a straightforward manner (e.g. API does not load, or the image does not load or there is no data). In multiple screens, the user would not be able to tell that something has gone wrong or that certain types of content have failed to load.

2) Match Between System and the Real World

- Strengths: The application uses familiar terms (e.g. Picture of the Day; Asteroid Tracker; Favourites) that most users will understand and use intuitively. All content on the application is sourced from NASA, therefore matches expectations of the user, as does the application design, which is based on a theme related to space.
- Weaknesses: Some of the scientific terminology and naming conventions used for asteroids, may be difficult for non-technical users to understand; e.g. the term "missions and news" is ambiguous without further context.

3) User Control and Freedom

- Strengths: Users can freely navigate through screens using consistent navigation patterns. There's direct user control provided with different features such as favourites and settings changes (e.g., dark mode, font size).
- Weaknesses: Limited abilities to undo, no fast navigation to return to the Home screen, and no filtering, sorting or resetting controls diminish user freedom.

4) Consistency and Standards

- Strengths: The app maintains consistency through the use of Material Design components, layouts (cards, lists, grids) and the same cohesive dark colour theme with purple accents. Patterns of interaction are the same on all screens. • Cons: There are minor inconsistencies in word usage or labelling (e.g., "Missions & News" vs. "NASA News") and visual themes (e.g., splash screen vs. main user interface). The design of some elements is slightly different than the original wireframes.
- Weaknesses: There are minor inconsistencies in word usage or labelling (e.g., "Missions & News" vs. "NASA News") and visual themes (e.g., splash screen vs. main user interface). The design of some elements is slightly different than the original wireframes.

5) Error Prevention

- Strengths: The design is built to limit user errors by reducing invalid inputs (e.g., using toggle buttons and pre-defining input choices) and allowing safe navigation. The data mechanism effectively handles preventing duplicate favourites.
- Weaknesses: Limited validation of API responses, missing data and broken image URLs. Minimal preventive feedback is offered (e.g., warnings prior to failures).

6) Recognition Rather Than Recall

- Strengths: Alternative visual method usage of features and contents is found using pictures, icons, card formats all help in the successful recognition of content you would like to engage with. Clear visual structures lead to less remembered prior actions.
- Weaknesses: Labelled content types to distinguish among favourite selections are missing some; saved items and history are not particularly easy to view.

7) Flexibility and Efficiency of Use

- Strengths: Navigation is simple with touch interactions for features providing a quick means of accessing key features. Efficient layouts (i.e. grids, lists) assist users in fast browsing.
- Weaknesses: Lacks more advanced functionality; no filters for searching, sorting options, gesture shortcut settings, customization for the usage of program provide inefficient use by familiar users and those that already know how to navigate the program.

8) Aesthetic and Minimalist Design

- Strengths: Excellent example of a well-designed application – modern (dark) theme, clean layouts, high quality pictures. Minimal clutter allows users to focus on the content especially in visually driven screens.
- Weaknesses: There are occasional instances of poor readability because of poor contrast and long lines of text. Some screens could improve on spacing and group related visuals better.

9) Help Users Recognise, Diagnose, and Recover from Errors

While the Celestia mobile app has a very visually appealing design with pleasing use of space in its interface and the intuitive nature of its navigation, there are several opportunities for improvement in the areas of error handling, user guidance, and advanced interaction functionality as described below:

- Strengths: Simple layouts and predictable navigational paths decrease the likelihood of making a mistake as a user. Automatic fall backs (e.g. splash screens) assist in preventing crashing of applications.
- Weaknesses: The absence of clear error messages, options to retry performing a specific action and guidance on how to recover from a failure (e.g. due to network connectivity, not having data, etc.) leaves users unsure of how to recover from their experiences.

10) Help and Documentation

- Strengths: The interface and design are intuitive and easy to learn and operate with minimal instruction for novices. The "About" section contains some basic information regarding the application.
- Weaknesses: There is no on boarding feature, help section, tool tips, or other choices for guidance. Based on the current state of those features, new users may not know how to utilize certain features such as their favourite items or advanced functionality.

Summary of Overall Findings:

Overall, the Celestia mobile app has strong usability with respect to visual design, presence of consistent elements, and ease of operation. Overall, the Celestia mobile app has strong usability with respect to visual design, presence of consistent elements and ease of operation. If the above areas were improved upon, it would significantly enhance the overall user experience and move the Celestia mobile app closer to becoming a production capable solution.

References:

- Apple. (2025) *Accessibility Human Interface Guidelines*. Apple Inc. Available at: <https://developer.apple.com/design/human-interface-guidelines/accessibility>.
- W3.org. (06 May 2025) *Mobile Accessibility At W3C*. Available at: [Mobile Accessibility at W3C | Web Accessibility Initiative \(WAI\) | W3C](#).
- Apple. (2025) *Human Interface Guidelines*. Apple Inc. Available at: <https://developer.apple.com/design/human-interface-guidelines/ios>.
- W3.org. (2025) *Web Content Accessibility Guidelines (WCAG)*. Available at: <https://www.w3.org/WAI/standards-guidelines/wcag/>.
- Nielsen, J. (1990) *Designing User Interfaces for International Use*. Available at: <https://www.scirp.org/reference/referencespapers?referenceid=2547451>.
- Donald A. Norman (01 April 1983) *Design Rules Based on Analyses of Human Error*. Available at: <https://dl.acm.org/doi/10.1145/2163.358092>.